AR HealthCare – Architecture & Design Overview

This document provides a structured, ODD-style overview of the architecture developed for the AR HealthCare system, including modules, patterns, dependencies, and runtime flow.

---

1. Design Patterns Used

1.1 Bridge Pattern

ARFCameraBridge implements a Bridge between:

- AR Foundation camera APIs (ARCameraManager / XRCpuImage / GPU textures)
- A unified output (`CurrentCameraTexture`) exposed to MediaPipe

This decouples platform camera details from the rest of the system.

1.2 Adapter Pattern

ARFImageSource adapts ARFCameraBridge to the MediaPipe `ImageSource` interface.

MediaPipe sees a standard `ImageSource`, but the actual frames come from AR Foundation.

1.3 Strategy Pattern

The `IBodyTracker` interface defines the tracking strategy.

Implementations:

- `MediaPipeTracker` (real body tracking)
- `MockBodyTracker` (synthetic pose data for testing)

The active strategy is injected at runtime.

1.4 Facade Pattern

TrackingManager acts as a Facade:

- hides complexity of trackers
- exposes one simple API: `GetPose()` via `ITrackingProvider`

Other modules only depend on the facade, not on specific trackers.

1.5 Dependency Inversion (DI)

Instead of referencing concrete classes:

- Consumers depend on `ITrackingProvider`
- TrackingManager depends on `IBodyTracker`
- SceneTrackerSetup chooses which implementation to inject

This increases modularity and testability.

1.6 Observer Pattern

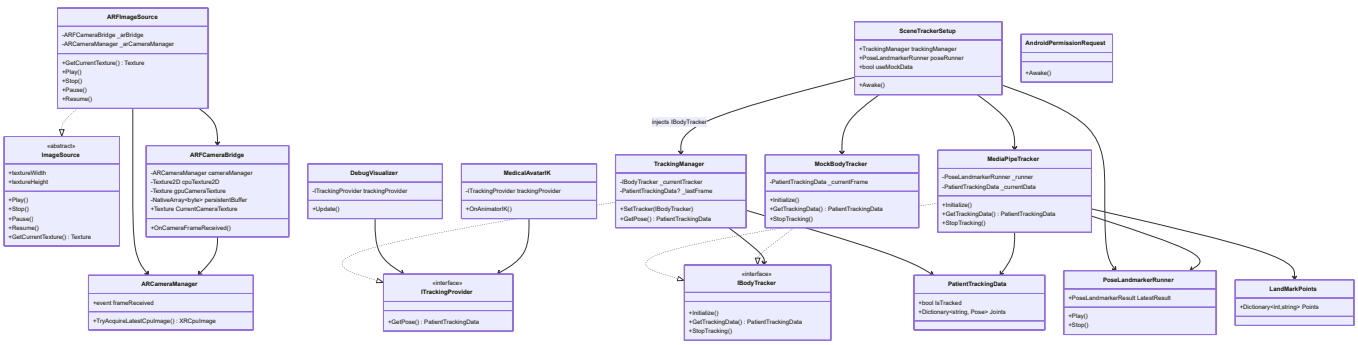
AR Foundation triggers camera updates using:

```
ARCameraManager.frameReceived
```

ARFCameraBridge subscribes and reacts when new frames arrive.

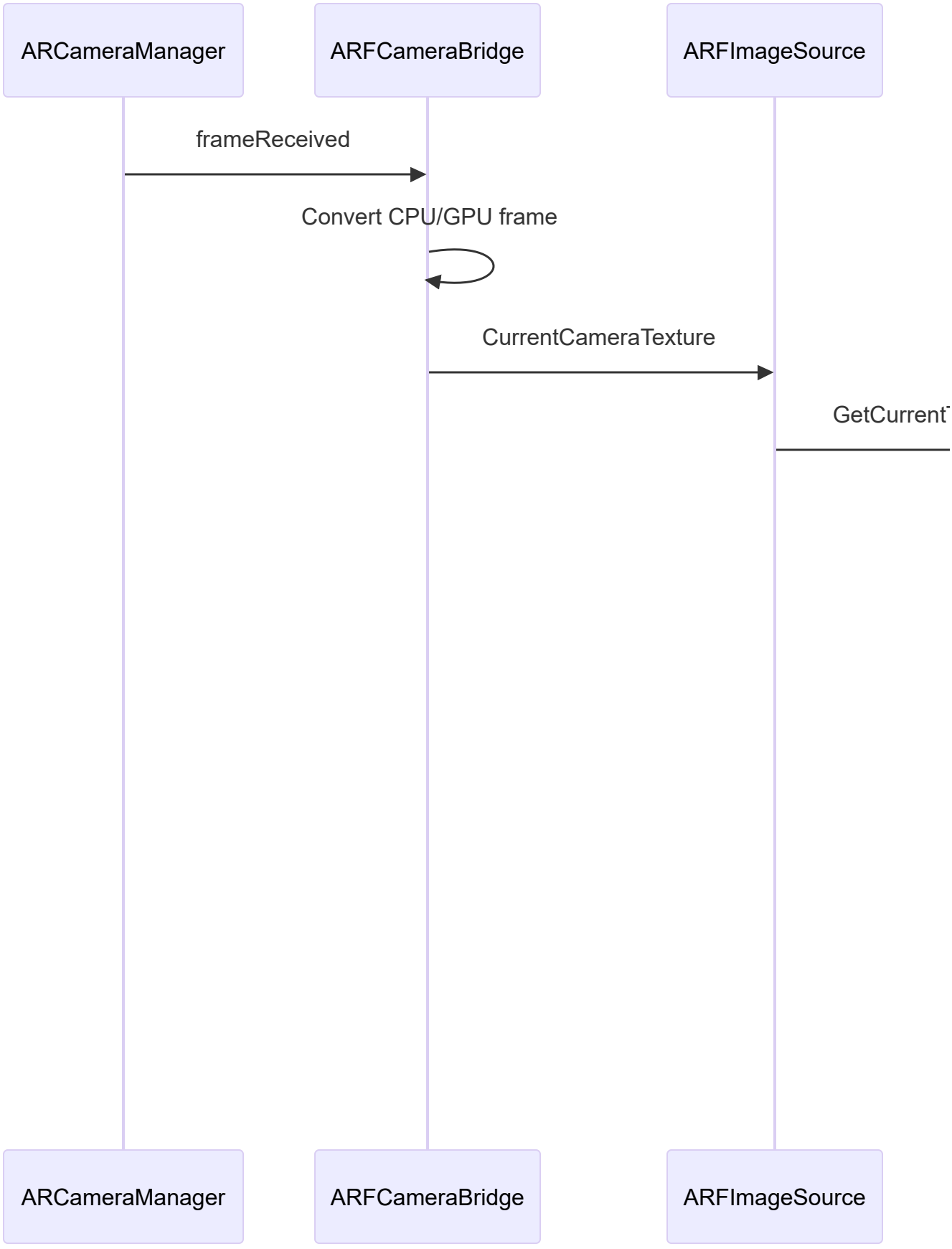
---

2. Dependency Graph (Class-Level)



---

3. Runtime Flow (Frame Lifecycle)



4. Module Overview

4.1 Input Pipeline

- **ARCameraManager** → produces raw camera frames (CPU/GPU)
- **ARFCameraBridge (Bridge)** → converts to unified `Texture`
- **ARFImageSource (Adapter)** → exposes frames to MediaPipe Engine

4.2 Tracking Pipeline

- **PoseLandmarkerRunner** → MediaPipe inference
- **MediaPipeTracker (Strategy)** → converts landmarks → `PatientTrackingData`
- **MockBodyTracker (Strategy)** → synthetic tracking
- **TrackingManager (Facade + Provider)** → central access point

4.3 Consumer Layer

- **MedicalAvatarIK** → applies tracked joints to the avatar
- **DebugVisualizer** → displays specific joints for testing

4.4 Setup Layer

- **SceneTrackerSetup** → injects the chosen tracking strategy
- **AndroidPermissionRequest** → handles camera permissions

---

5. Responsibilities Summary

Component	Responsibility	Pattern
ARFCameraBridge	Convert AR frames to Texture	Bridge
ARFImageSource	Adapt Texture to MediaPipe ImageSource	Adapter
IBodyTracker	Tracking algorithm interface	Strategy
MediaPipeTracker	Real tracking from MediaPipe	Strategy impl
MockBodyTracker	Mock tracking	Strategy impl
TrackingManager	Single access point for tracking data	Facade + DI
ITrackingProvider	Interface exposing GetPose()	DI abstraction
MedicalAvatarIK	Consumes tracking data to drive avatar	Consumer
SceneTrackerSetup	Injects tracker into manager	Dependency Injection

6. High-Level Architecture Chart

```
DEVICE → AR FOUNDATION → BRIDGE → IMAGE SOURCE → MEDIAPIPE → TRACKER →
MANAGER → AVATAR
```

This document is ready for inclusion in an ODD-style architecture document or software design review.